

Learning Tracker

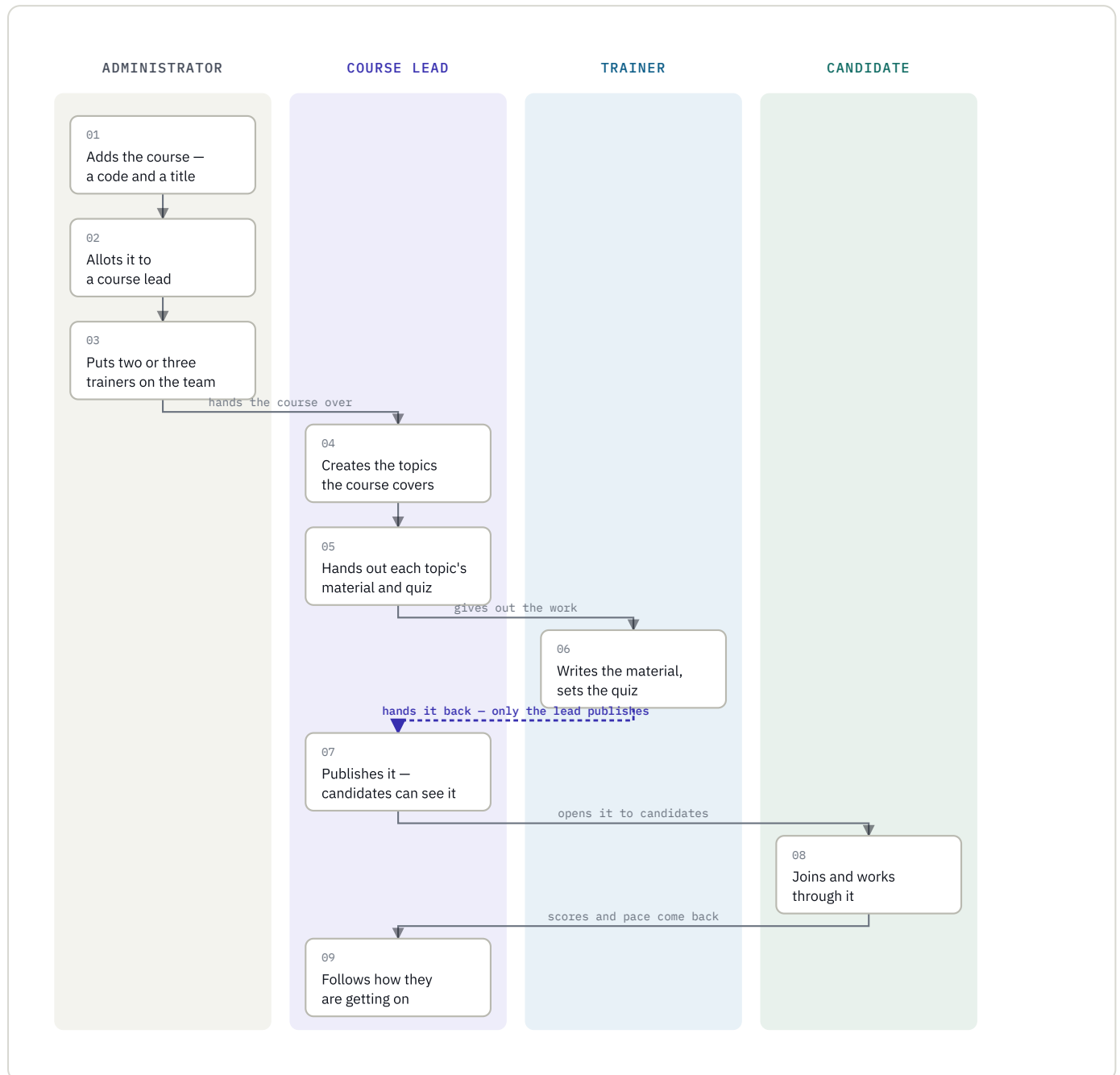
An internal training platform. Administrators set up courses and staff them; a course lead runs each one and divides the writing among their trainers; candidates work through the topics released to them and sit the quizzes.

Stack Node · Express · Prisma · PostgreSQL · React · Vite · Tailwind

Roles 4 Migrations 10 Status in development

How it works

A course passes through four pairs of hands. Nobody does the next person's job: the administrator decides what exists and who staffs it, the lead decides what goes in it and who writes each part, the trainers write, and only the lead makes any of it visible.



Each column is a person, each row a step. Work crosses a column only where one job ends and another begins — and the dashed line is the gate: a trainer writes a topic but can never release it.

Step 08 has its own small loop. A candidate either asks to join a published course and waits for the lead to approve them, or the lead allots them topics directly, which enrolls them at the same time. Either way they see only the topics released to them — never the whole course.

The four roles

A role is on the account, and an account has exactly one. Lead and trainer are different jobs rather than ranks — moving between them is a deliberate change on the administration screen, never a side effect of being given work.

ADMINISTRATOR

Creates courses, decides who leads each one and who is on its team. Sees every course and every person. Cannot be given a course to run.

COURSE LEAD

Owns the courses allotted to them. Creates topics, hands out the work, approves candidates, and is the only one who can publish anything.

TRAINER

Works under leads. Writes the material or sets the quiz for the topics handed to them. Never owns a course and never publishes.

CANDIDATE

Attends courses. Reads the material for topics released to them and sits the quizzes. Sees nothing that has not been published and allotted.

Authority is still checked *per course*. Leading one course grants nothing on another, and a trainer on one team has no standing on a course they are not part of.

Who can do what

Every rule below is enforced server-side in `courses.service.js`, which is the only place these decisions are made. The screens reflect it — they never re-derive it.

ACTION	ADMIN	LEAD	TRAINER ON DUTY	TRAINER OTHER TOPIC
Create a course	yes	—	—	—
Change a course code	yes	—	—	—
Allot a course to a lead	yes	—	—	—
Put trainers on the team	yes	—	—	—
Edit title, description, duration	yes	yes	—	—
Create and delete topics	yes	yes	—	—
Hand out material and quiz duty	yes	yes	—	—
Upload and remove material	yes	yes	material only	—
Write quiz questions	yes	yes	quiz only	—
Publish course, topic or quiz	yes	yes	—	—
Approve join requests	yes	yes	—	—
Allot topics to candidates	yes	yes	—	—
Remove a candidate from the course	yes	yes	—	—
Read the course, its progress and feedback	yes	yes	yes	yes

The two **partial** cells are the point of the split: a topic is two jobs. Holding the material duty does not let you touch the quiz, and holding the quiz duty does not let you touch the files.

The checks themselves

FUNCTION	PASSES FOR	USED BY
<code>assertCourseRead</code>	admin, lead, anyone on the team	course view, progress, feedback
<code>assertCourseLead</code>	admin, lead	course edits, topics, candidates
<code>assertTopicRead</code>	admin, lead, anyone on the team	quiz view, material download
<code>assertMaterialWrite</code>	admin, lead, material-duty trainer	material upload and delete
<code>assertQuizWrite</code>	admin, lead, quiz-duty trainer	quiz and question writes
<code>assertTopicLead</code>	admin, lead	publishing, duty, topic deletion
<code>assertNotCourseStaff</code>	—	blocks enrolling staff as learners
<code>assertNotEnrolled</code>	—	blocks staffing an enrolled learner

Rules the system enforces

These hold regardless of which screen or endpoint you come through.

- **Only a lead owns a course.** Allotting one to a trainer is refused; naming a candidate makes them a lead.
- **Only a trainer sits on a team.** Adding a lead is refused; naming a candidate makes them a trainer.
- **Staff are not learners on their own course.** Enrolling the lead or a team trainer is refused, and so is staffing somebody already enrolled.
- **Publishing belongs to the lead.** Course, topic and quiz alike — the check is separate from the rest of the update, so a trainer can change a quiz's pass mark but not release it.
- **A course cannot lose its lead.** Changing their role is refused while they still run something; the same for a trainer still on a team.
- **Candidates see only what was allotted.** Topic access is granted row by row, and a course must be published to appear in the catalogue at all.
- **The latest scored attempt counts,** not the best. Retakes are kept and shown, marked as superseded.
- **Answered questions are frozen.** Editing or deleting one after a candidate has answered is refused, so stored scores always match the wording they were earned against.
- **Course codes are unique** and reported as a conflict rather than a database error.

Data model

Content nests: a **course** holds **topics**, a topic holds **material** and at most one **quiz**, and a quiz holds **questions** with their options. Everything else attaches people to that structure.

MODEL	HOLDS	NOTES
User	email, name, role, active flag	One role per account.
Course	code, title, description, duration, published	<code>ownerId</code> is the lead, and is nullable — a course exists before it is allotted.
CourseTrainer	course + trainer	The team. The lead is not a row here.
Topic	position, title, description, published	Two duty columns: <code>materialTrainerId</code> , <code>quizTrainerId</code> .
Material	type, title, file key, size	Uploaded PDFs and slides; stored on disk under <code>uploads/</code> .
Quiz	title, pass mark, max attempts, published	At most one per topic.
Question	prompt, marks, type, options	Single- or multi-answer, decided by how many options are correct.
Enrollment	candidate + course, status, dates	<code>pending</code> until approved; carries started and completed timestamps.
TopicAssignment	candidate + topic, who granted it	The row that actually reveals a topic to a candidate.
Attempt	attempt number, score, percentage, submitted	Every retake kept; the latest scored one counts.
Answer	marks awarded, correctness, selected options	All-or-nothing marking, no partial credit.
CourseFeedback	rating 1–5, comment	One per candidate per course, editable.

Marking rule. A question earns its marks only if the selected options exactly match the correct set — every right one ticked, no wrong ones. Course averages are marks-weighted, so a ten-mark quiz counts for more than a two-mark one.

API reference

Seven routers under `/api`. Every route needs a bearer token; the role listed is the router-level gate, and per-course checks run inside on top of it.

Authentication `/api/auth`

ROUTE	WHO	DOES
POST <code>/register</code>	public	Self sign-up. Always creates a candidate.
POST <code>/login</code>	public	Returns a token and the user.
GET <code>/me</code>	any	Current user, re-read from the database each call.
POST <code>/users</code>	admin	Creates an account of any role.

Administration `/api/admin` • admin only

ROUTE	DOES
GET <code>/overview</code>	Everything for the admin screens in one payload: courses with teams, leads, trainers, candidates, administrators, headline stats.
GET <code>/candidates/:userId</code>	One person in full: courses, per-course progress, every attempt.
PATCH <code>/users/:userId/role</code>	Moves someone between candidate, trainer and lead.
POST <code>/courses</code>	Creates a course. The lead is optional at this point.
POST <code>/allotments</code>	Hands a course to a lead, promoting a candidate if needed.
POST <code>/courses/:courseId/team</code>	Puts a trainer on the team.
DELETE <code>/courses/:courseId/team/:userId</code>	Takes them off, clearing any duties they held.

Courses and topics `/api/courses` • trainer, lead, admin

ROUTE	DOES
<code>GET /</code>	Courses you lead or are on the team of, with your standing on each.
<code>GET /:courseId</code>	The course with topics, duties, team and what the viewer may do.
<code>PATCH /:courseId</code>	Edits the course. Only an admin may send a new code.
<code>DELETE /:courseId</code>	Deletes it and everything under it.
<code>GET /:courseId/progress</code>	Every candidate: scores, pace, weak topics.
<code>GET /:courseId/feedback</code>	Ratings and comments with their distribution.
<code>POST /:courseId/topics</code>	Appends a topic.
<code>PATCH /topics/:topicId</code>	Renames or publishes a topic.
<code>PATCH /topics/:topicId/duty</code>	Hands out the material and quiz halves — send either or both.
<code>DELETE /topics/:topicId</code>	Removes a topic.

Material `/api/content`

ROUTE	DOES
<code>POST /topics/:topicId/materials</code>	Uploads a file, up to 50 MB.
<code>GET /materials/:materialId/file</code>	Streams it. Candidates need the topic allotted; staff need to be on the course.
<code>DELETE /materials/:materialId</code>	Removes the record and the file.

Quizzes `/api/quizzes` • trainer, lead, admin

ROUTE	DOES
<code>POST /topics/:topicId</code>	Fetches the topic's quiz, creating an empty one on first use.
<code>GET /:quizId</code>	The quiz with its questions and attempt count.
<code>PATCH /:quizId</code>	Title, pass mark, attempt limit — and publishing, which is lead-only.
<code>POST /:quizId/questions</code>	Adds a question with its options.
<code>PATCH /questions/:questionId</code>	Rewrites one, refused once anybody has answered it.
<code>DELETE /questions/:questionId</code>	Removes one, under the same condition.
<code>POST /:quizId/import</code>	Reads questions out of a PDF or Word file and returns them for review. Saves nothing.
<code>POST /:quizId/questions/bulk</code>	Saves the reviewed ones.

Allotment and requests `/api/allot` • trainer, lead, admin

ROUTE	DOES
<code>GET /candidates</code>	Active candidates, for the picker.
<code>GET /requests</code>	The inbox — pending requests across the courses you lead.
<code>GET /topics/:topicId/assignments</code>	Who can currently open a topic.
<code>POST /topics/:topicId/assignments</code>	Releases one topic to candidates, enrolling them if needed.
<code>DELETE /topics/:topicId/assignments/:userId</code>	Withdraws that topic.
<code>POST /courses/:courseId/assignments</code>	Releases every topic at once.
<code>POST /courses/:courseId/requests/:userId/approve</code>	Approves a request, optionally allotting everything.
<code>DELETE /courses/:courseId/requests/:userId</code>	Turns a request down.
<code>DELETE /courses/:courseId/enrollments/:userId</code>	Takes a candidate off the course and withdraws their topics. Attempts are kept.

Learning `/api/learn` • candidate only

ROUTE	DOES
<code>GET /catalogue</code>	Every published course, with whether you have asked to join.
<code>GET /my-courses</code>	Your courses and the topics released to you.
<code>GET /progress</code>	Your scores across everything.
<code>POST /courses/:courseId/subscribe</code>	Asks to join. Creates a pending request, never instant access.
<code>DELETE /courses/:courseId/subscribe</code>	Withdraws a request that has not been approved.
<code>GET /topics/:topicId</code>	One topic's material.
<code>GET /topics/:topicId/quiz</code>	The quiz and how many attempts remain.
<code>POST /topics/:topicId/quiz/attempts</code>	Sits and submits an attempt; scored on the spot.
<code>GET /attempts/:attemptId/review</code>	What you answered against what was right.
<code>GET /courses/:courseId/feedback</code>	Your rating and comment.

Screens

What each role sees after signing in. Navigation differs by role; the route table is the same app.

PATH	SCREEN	WHO
/admin	Administration — leads, trainers, candidates, administrators	admin
/admin/courses	Course catalogue — add a course, rename or re-code one	admin
/admin/allotment	Allotment — allot a lead, manage each course's team	admin
/admin/candidates/:id	One candidate in full, with every attempt	admin
/trainer	My courses — what you lead or are on the team of	trainer, lead, admin
/trainer/courses/:id	The course: topics, duties, material, quizzes, access	trainer, lead, admin
/trainer/courses/:id/progress	Candidate progress, completion and pace	trainer, lead, admin
/trainer/inbox	Join requests waiting on you	lead, admin
/home · /browse · /my-courses · /my-progress	The candidate's four screens	candidate

Schema history

Ten migrations. The last five are where the staffing model took shape.

MIGRATION	CHANGED
init	Users, courses, topics, material, quizzes, attempts.
enrollment_status	Pending versus active places on a course.
course_feedback	Ratings and comments.
enrollment_dates	Started and completed timestamps.
course_duration	Expected length in weeks.
course_owner_optional	A course may exist before it has a lead.
course_team_and_topic_duty	Course teams, and a trainer on duty per topic.
add_lead_role	<code>lead</code> joins the role enum.
existing_owners_become_leads	Data migration: every course owner promoted to lead.
split_topic_duties	One duty becomes two — material and quiz — carrying the old value into both.

Two of these touch data, not just shape. `existing_owners_become_leads` promotes accounts, and `split_topic_duties` copies each existing duty into both new columns. Anyone deploying will see roles change — that is intended.

Running it

Two processes. The front end proxies `/api` to the back end, so no CORS setup is needed in development.

WHERE	COMMAND	GIVES
backend	<code>npm start</code>	API on :4000
backend	<code>npm run migrate</code>	Applies migrations
backend	<code>npm run seed</code>	Demo accounts and a sample course
backend	<code>npm run studio</code>	Prisma's database browser
frontend	<code>npm run dev</code>	App on :5173

The seed creates one administrator, a lead with a five-topic course, a handful of trainers and a list of candidates. Every seeded account shares one password, taken from `SEED_PASSWORD` and defaulting to a development value in `prisma/seed.js`. Re-running the seed never changes anybody's role, so accounts promoted since are left alone.

HONEST LIST

Known gaps

Things the system does not do yet, roughly in the order worth fixing.

- **Accounts cannot be deactivated.** The active flag is read everywhere and written nowhere, so a leaver cannot be switched off.
- **Deleting a course or topic leaves its files behind.** Cascades remove the records; only the material-delete route removes the bytes.
- **No tests, no CI.** The permission matrix above is verified by hand. It is the part of the system most worth pinning down automatically.
- **Two dead schema values.** In-app text material and written answers exist in the enums with nothing behind them — build or drop.
- **No screen for creating accounts.** Administrators are minted through the API only.
- **Nothing notifies anybody.** Approvals, duties and waiting requests are discoverable only by looking.
- **No audit trail.** Who published a course, and when, has no answer.
- **The admin overview loads everything** — every user, course and scored attempt, on each visit. Fine at this size; not at scale.
- **Single sign-on is still ahead.** Real staff are expected to come from the office directory, which retires the shared seed password.

Learning Tracker — internal documentation. Generated from the codebase as it stands: the route tables, permission checks and migration list above were read from source rather than recalled, so they match what runs.